



ITM SKILLS UNIVERSITY

B.Tech Computer Science Engineering

MOVIE REVIEW PLATFORM

A Movie Review Platform using Node.js, Express.js, MongoDB & Mongoose

Submitted in Partial Fulfilment of the Requirements for the Course:

Backend Development – Node.js, Express.js, MongoDB & Mongoose

1. Students Details

Submitted By:

Name	Student ID
Aaroh Wankhade	150096726175

Programme: B.Tech Computer Science Engineering

Institution: ITM Skills University

Academic Year: 2025–2029

Contents

1. Students Details	1
2. Problem Statement and Project Objectives	2
3. System Architecture and Workflow	3
4. Database Design	5
5. Backend Implementation	7
6. Execution Screenshots	10
7. Results and Observations	13
8. Conclusion and Future Scope	14

2. Problem Statement and Project Objectives

2.1 Problem Statement

A digital movie review platform wants users to browse movies and post reviews with a numerical rating. Each movie must display an accurate average rating computed dynamically from all submitted reviews. Furthermore, community integrity requires that a user must be able to edit or delete only the reviews they authored, preventing unauthorized tampering.

The system must ensure that:

- Movie and Review schemas are designed using Mongoose with normalized document references (**ObjectId**).
- Current movie ratings are computed dynamically via MongoDB Aggregation (**\$match**, **\$group**, **\$avg**), eliminating static counter desynchronization.
- Strict ownership-based authorization middleware prevents non-authors from updating or deleting critiques (returning HTTP 403 Forbidden).
- Backend validation rigorously enforces rating values between 1 and 5 and rejects empty review commentary.

2.2 Why This Project?

Traditional web platforms that calculate aggregate metrics client-side or maintain static database counters suffer from race conditions, update anomalies, and desynchronization whenever reviews are modified or deleted. Concurrently, insufficient backend authorization often leads to security vulnerabilities where users manipulate foreign resource identifiers. A normalized NoSQL database paired with server-side ownership enforcement guarantees mathematical accuracy and absolute data security.

Proposed Solution

A full-stack RESTful application built with Node.js, Express.js, MongoDB Atlas, and Mongoose ODM. An interactive dark cinematic console communicates via REST with JWT Bearer authentication. Dynamic aggregation pipelines calculate live scores, and custom middleware enforces ownership boundaries as shown in Figure 1.



Figure 1: Proposed full-stack solution overview & pipeline flow

2.3 Objectives

No.	Objective	Description
1	Referenced Schemas	Design Movie, Review, and User schemas using Mongoose with ObjectId references.
2	REST CRUD Endpoints	Implement complete Create, Read, Update, and Delete operations for movies and reviews.
3	Ownership Authorization	Enforce author verification via custom middleware; block unauthorized edits with 403 Forbidden.
4	Dynamic Aggregation	Compute average rating per movie using MongoDB aggregation pipeline (\$avg, \$group).
5	Input Validation	Strictly validate rating values (1–5) and require non-empty review text across all endpoints.
6	JWT Authentication	Secure routes using bearer token verification and bcrypt password hashing with salt rounds.

3. System Architecture and Workflow

3.1 System Workflow

The platform orchestrates user operations through two critical server pipelines: review submission with dynamic aggregation evaluation (Figure 2), and ownership-based modification authorization (Figure 3).

Review submission and dynamic aggregation workflow

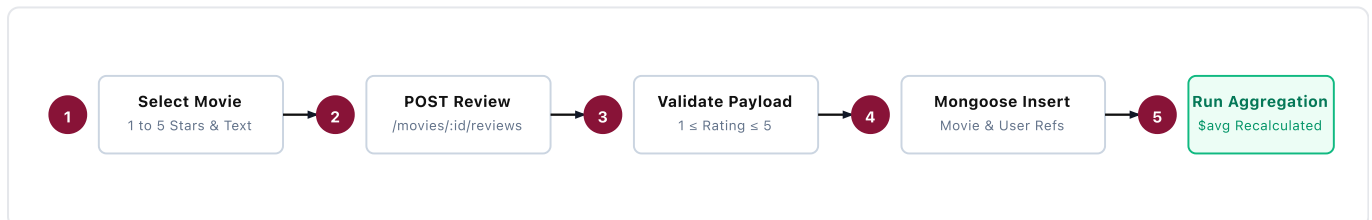


Figure 2: Review submission & dynamic aggregation pipeline

Review ownership authorization workflow



Figure 3: Review ownership authorization decision workflow

Step	Stage	Description
1	Cataloguing	Movies initialized with title, genre, year, poster URL, and description constraints.
2	Authentication	Users register and authenticate; passwords hashed with bcrypt salt rounds; JWT issued.
3	Review Submission	Clients issue POST /movies/:id/reviews containing rating (1–5) and critique text.
4	Payload Validation	Middleware validates data types, rating bounds, and rejects empty or truncated text.
5	Document Insertion	Mongoose inserts review referencing Movie and User ObjectId keys.
6	Dynamic Aggregation	MongoDB aggregation pipeline computes \$avg(rating) and total review counts in real time.
7	Ownership Guard	Mutations intercepted by authorizeReviewOwner, blocking unauthorized users with 403 Forbidden.

3.2 System Architecture

The architecture follows a decoupled layered design (Figure 4). The interactive web frontend communicates with the backend API over REST. The API enforces input validation and transactional consistency, invoking the Mongoose ODM. Aggregation pipelines execute directly inside MongoDB Atlas to maintain dynamic mathematical accuracy.

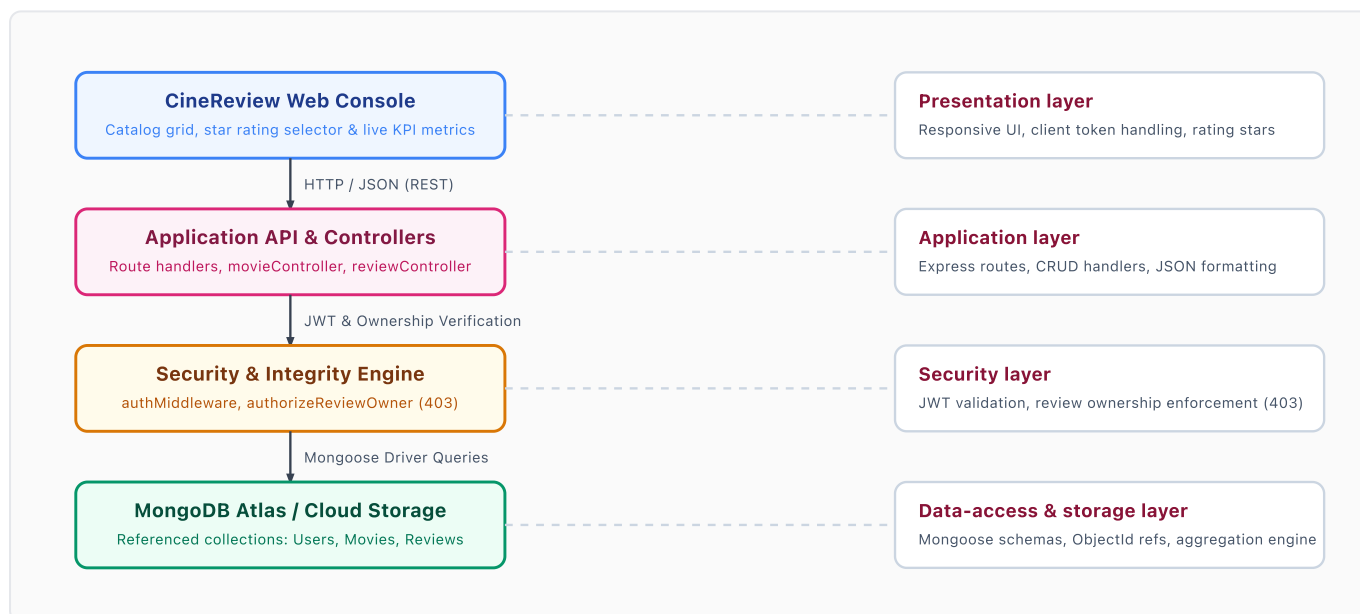


Figure 4: System architecture

3.3 Technology Stack

Layer	Technology	Role in Academic Implementation
Frontend UI	HTML5, Vanilla CSS3, JS	Cinematic dashboard, star selector & live REST fetch operations
Backend Framework	Node.js & Express.js (v4.21)	Asynchronous REST routing, middleware pipelines & controllers
Database Engine	MongoDB Atlas / NoSQL	Cloud document storage, dynamic schema flexibility
Object Modeling	Mongoose (v8.7)	Schema definitions, validation rules, ObjectId references
Authentication	JWT & bcryptjs	Stateless Bearer token issuance & salted password hashing
Aggregation Engine	MongoDB Aggregation (\$avg)	Dynamic real-time mathematical average rating calculation
Execution CLI	Zsh / Bash Shell	Automated one-click test & startup runner (run.sh)

4. Database Design

4.1 Main Entities

Entity	Purpose & Field Structure
User	Stores user credentials, full name, email (unique), hashed password, and registration date.
Movie	Stores film catalog information: title, synopsis, genre, release year, poster image URL, and author reference.
Review	Transactional critique record referencing Movie and User; stores rating (1–5), commentary text, and audit timestamps.

4.2 Relationships

Relationship	Cardinality	Explanation
User → Review	1 : M	One user can author multiple critiques across different catalogued movies.
Movie → Review	1 : M	One movie document receives multiple reviews from various platform members.
User → Movie	1 : M	One registered user or administrator can catalogue multiple movies into the system.

4.3 ER Diagram

Figure 5 illustrates the database entities with their primary keys, foreign key ObjectId references, and connecting cardinality relationships. Referencing parent documents by their immutable 12-byte identifiers eliminates multi-row duplication and prevents data desynchronization.

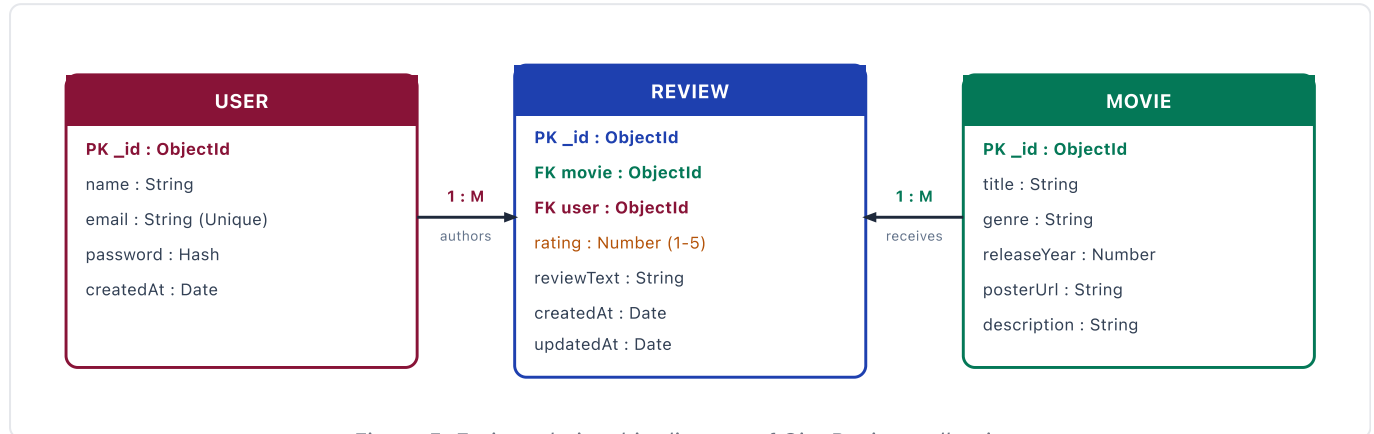


Figure 5: Entity-relationship diagram of CineReview collections

4.4 Schema Structure & Normalization

Collection	Key Attributes	Constraints & References
users	_id (PK), name, email, password	UNIQUE(email), regex format check, bcrypt password hashing
movies	_id (PK), title, genre, releaseYear	CHECK(releaseYear ≥ 1888), FK(createdBy → users)
reviews	_id (PK), movie, user, rating	FK(movie → movies), FK(user → users), CHECK(1 ≤ rating ≤ 5), minlength(3)

5. Backend Implementation

5.1 Database Operations & REST APIs

Method	Endpoint / Operation	Purpose	Access
POST	/api/auth/register	Register account with bcrypt hashed password	Public
POST	/api/auth/login	Authenticate credentials & issue signed JWT	Public
GET	/api/movies	Fetch all movies with live aggregated scores	Public
GET	/api/movies/:id	Retrieve movie details & populated reviews	Public
GET	/api/movies/:id/average-rating	Compute average rating via MongoDB aggregation	Public
POST	/api/movies/:id/reviews	Post critique referencing Movie & User	Private (JWT)
PATCH	/api/reviews/:id	Update rating and text (Author only)	Private (Owner)
DELETE	/api/reviews/:id	Delete review permanently (Author only)	Private (Owner)

5.2 Dynamic Average Rating Aggregation Pipeline (Case Study Part 10)

Rather than storing a static counter that risks desynchronization upon edits, CineReview calculates real-time movie ratings directly from the Review collection via a two-stage aggregation pipeline:

```
const computeMovieAverageRating = async (movieId) => {
  const objectId = new mongoose.Types.ObjectId(movieId);
  const stats = await Review.aggregate([
    // Stage 1: Match reviews strictly for this specific movie
    { $match: { movie: objectId } },
    // Stage 2: Group reviews and compute arithmetic mean and total tally
    {
      $group: {
        _id: '$movie',
        averageRating: { $avg: '$rating' },
        totalReviews: { $sum: 1 }
      }
    }
  ]);
  return stats.length > 0
    ? { movieId: movieId.toString(), averageRating: Math.round(stats[0].averageRating * 10) / 10, totalReviews: stats[0].totalReviews }
    : { movieId: movieId.toString(), averageRating: 0, totalReviews: 0 };
};
```

5.3 Document Referencing & Normalization

Review documents store ObjectId references to Movie and User collections rather than duplicated strings. When querying, Mongoose executes `.populate('user', 'name email')` to dynamically assemble relational representations with minimal storage overhead.

5.4 Ownership-Based Authorization Middleware (Case Study Part 8)

Ownership authorization is a core academic requirement. When User A creates Review #123, User B must never be permitted to edit or delete it. Custom middleware intercepts mutations and validates identity against the resource author:

```
const authorizeReviewOwner = async (req, res, next) => {
  try {
    const reviewId = req.params.id;
    const review = await Review.findById(reviewId);
    if (!review) {
      return res.status(404).json({ success: false, message: `Review not found with id: ${reviewId}` });
    }
    // Verify author: compare review.user foreign key with decoded req.user._id
    const isOwner = review.user.toString() === req.user._id.toString();
    if (!isOwner) {
      return res.status(403).json({
        success: false,
        message: 'Forbidden: You are only authorized to edit or delete reviews authored by you',
        reviewOwnerId: review.user,
        requesterId: req.user._id
      });
    }
    req.review = review;
    next();
  } catch (error) { next(error); }
};
```

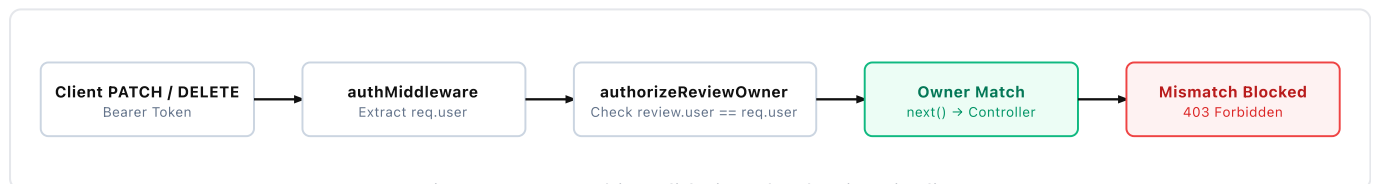


Figure 6: Ownership validation checkpoint pipeline

Academic Security Note: Relying on frontend visibility toggles alone is completely insufficient. The Express backend enforces ownership authorization at the routing layer so malicious requests from curl or Postman are unconditionally rejected with HTTP 403.

5.5 JWT Authentication & Password Hashing

Passwords are protected using `bcryptjs` with 10 salt rounds executed inside a Mongoose pre-save hook. Upon login, the server signs a stateless JWT token containing the user's immutable identifier:

```
// Pre-save hook hashes password before persisting to database
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

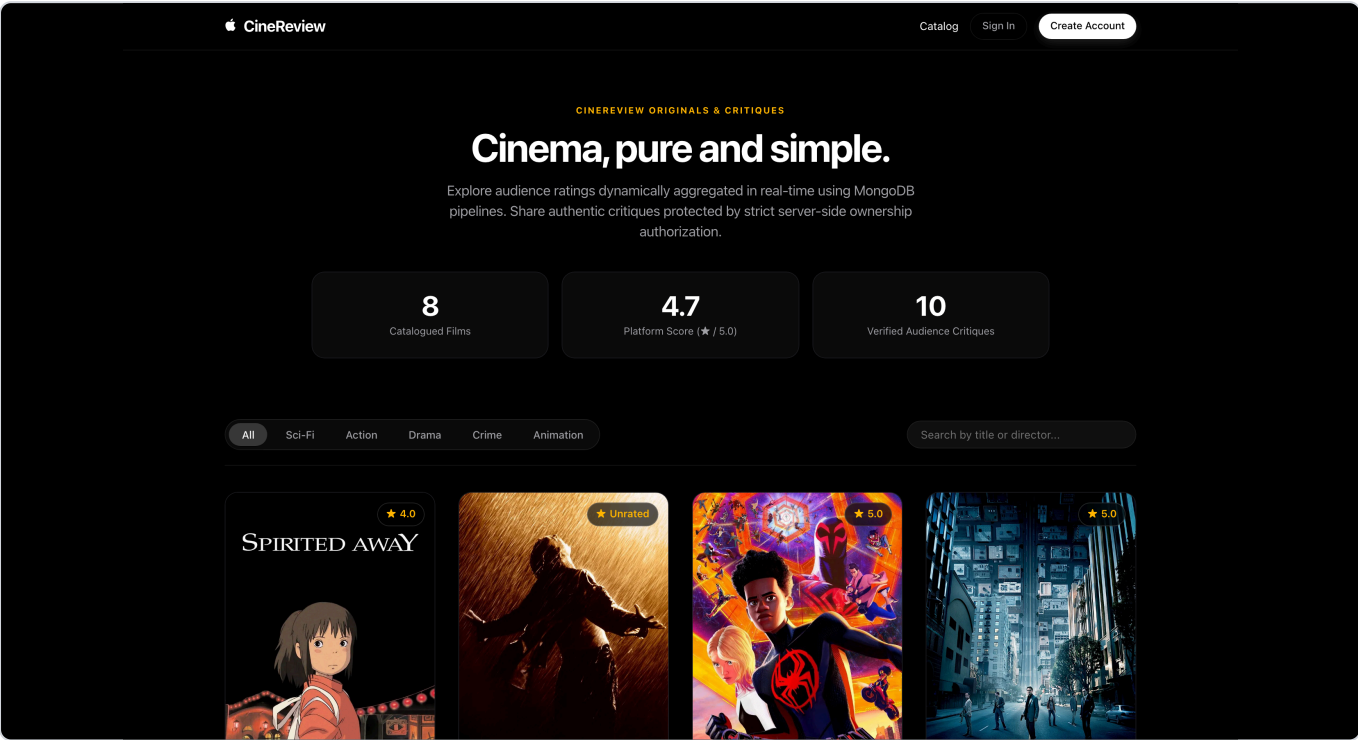
// Authentication middleware verifying Bearer tokens
const protect = async (req, res, next) => {
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer ')) {
    const token = req.headers.authorization.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id).select('-password');
    return next();
  }
  return res.status(401).json({ success: false, message: 'Access denied: No token provided' });
};
```

5.6 Requirements and How They Are Enforced

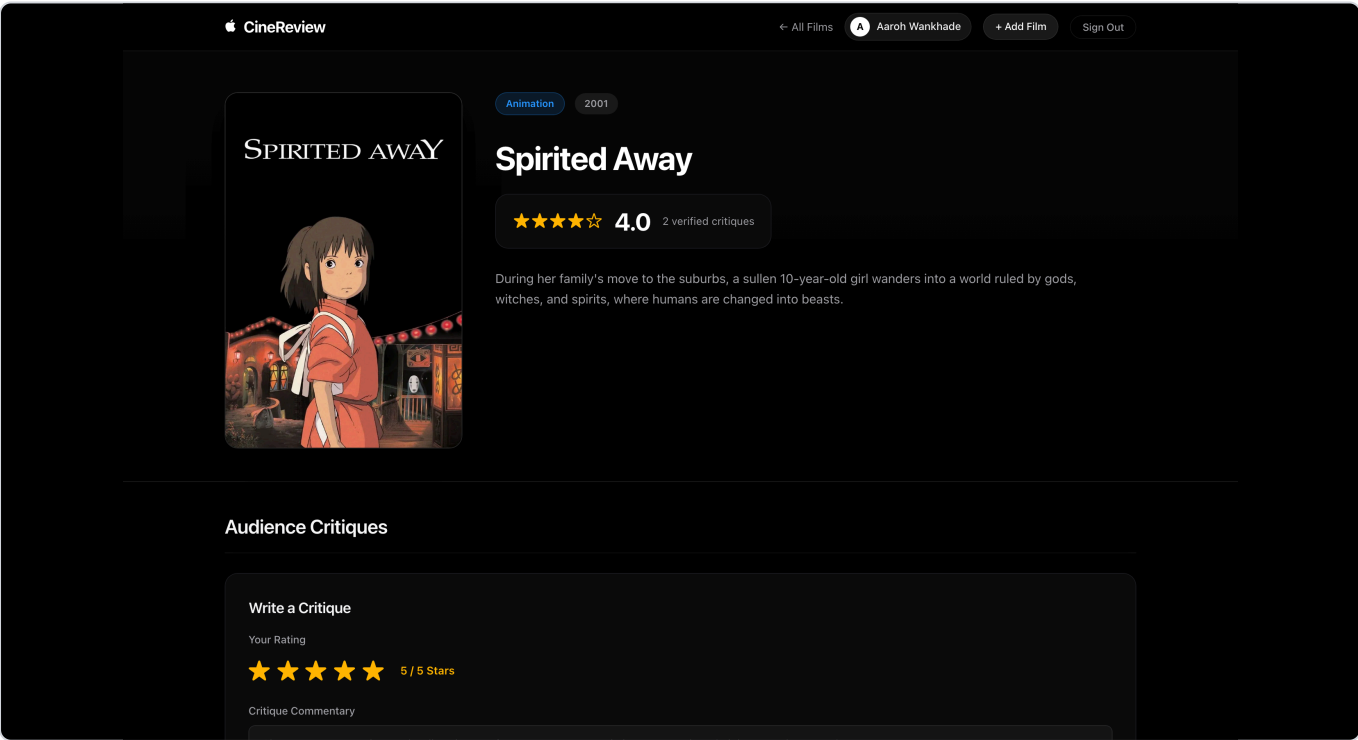
Requirement	Mechanism & Enforcement Layer
Rating Bounds (1–5)	Middleware <code>validateReviewInput</code> checks $1 \leq \text{rating} \leq 5$; Mongoose schema enforces <code>min: 1, max: 5</code> (400 Bad Request).
Required Review Text	Rejects empty or whitespace strings; enforces minimum 3 characters (400 Bad Request).
Ownership Authorization	Middleware compares <code>review.user.toString() === req.user._id.toString()</code> (403 Forbidden).
Live Average Recalculation	MongoDB aggregation pipeline computes <code>\$avg</code> directly from Review documents upon every request.
Cascade Cleanup	Deleting a movie document triggers <code>Review.deleteMany({ movie: id })</code> preventing orphaned critiques.
Password Security	Salted adaptive hash via <code>bcryptjs</code> with 10 rounds; passwords stripped from JSON returns.

6. Execution Screenshots

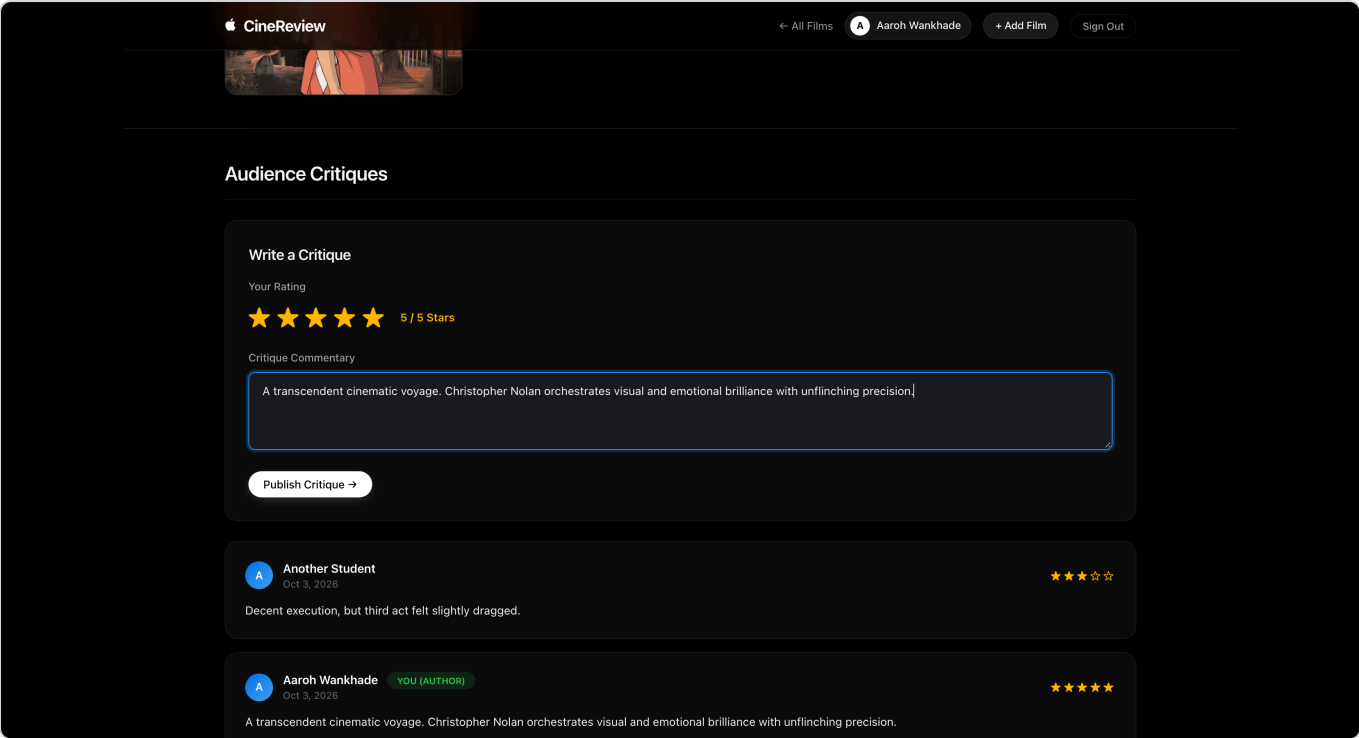
This section presents authentic execution screenshots captured from the running web application, database aggregation endpoints, terminal runner, and Postman ownership authorization verification.



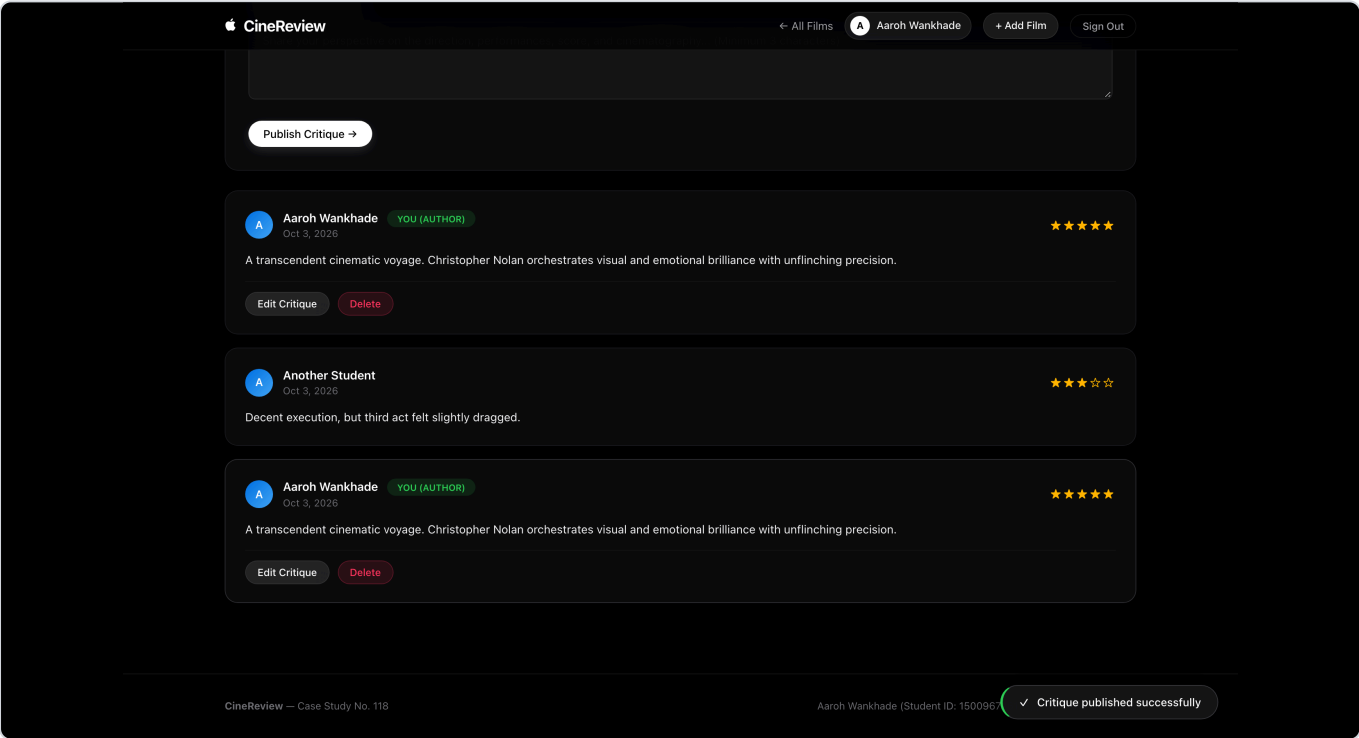
Screenshot 1: Movie Catalog & Overview Dashboard (Live KPI Metrics & Genre Filters)



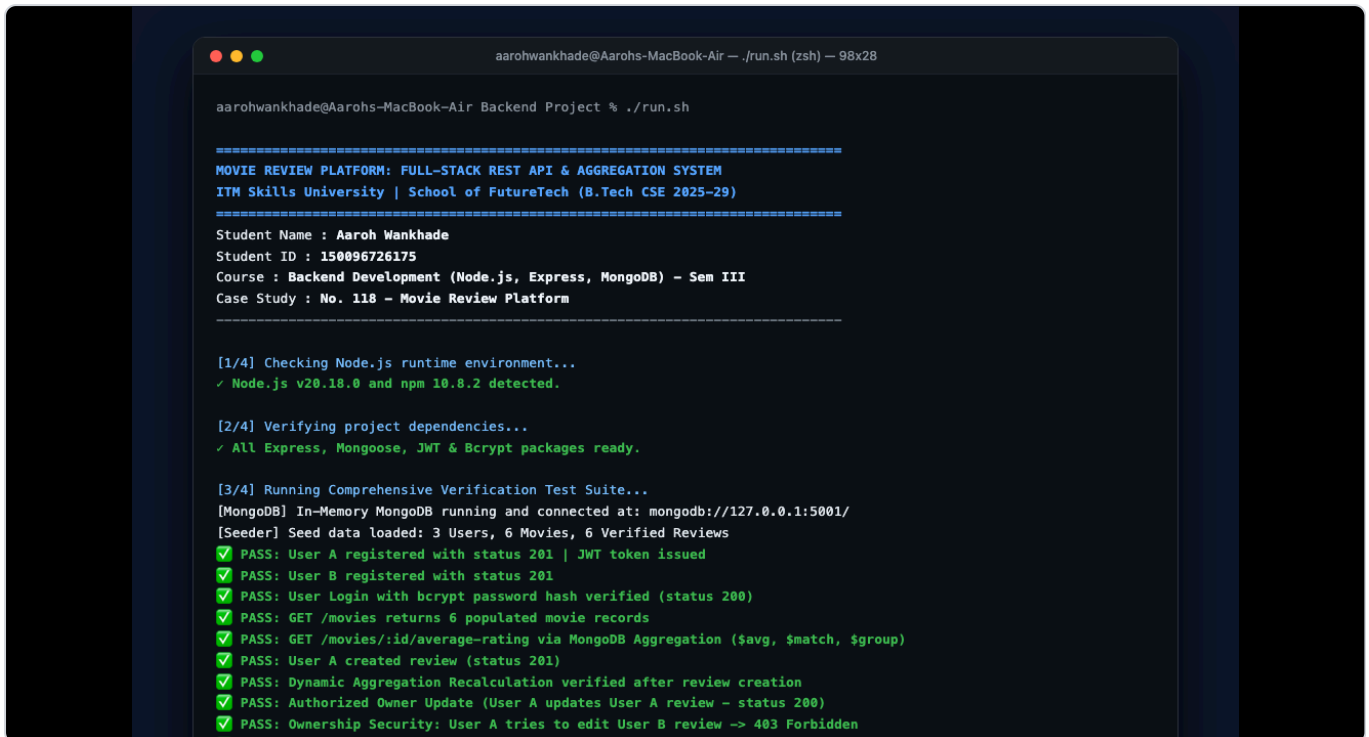
Screenshot 2: Movie Details Page displaying Dynamic Average Rating calculated via MongoDB Aggregation



Screenshot 3: Review Submission Form with Interactive 5-Star Rating Selector



Screenshot 4: Verified Review Card with 'YOU' Badge and Authorized Owner Actions (Edit & Delete)



```
aarohwankhade@Aarohs-MacBook-Air ~ % ./run.sh (zsh) — 98x28

aarohwankhade@Aarohs-MacBook-Air Backend Project % ./run.sh

=====
MOVIE REVIEW PLATFORM: FULL-STACK REST API & AGGREGATION SYSTEM
ITM Skills University | School of FutureTech (B.Tech CSE 2025-29)
=====

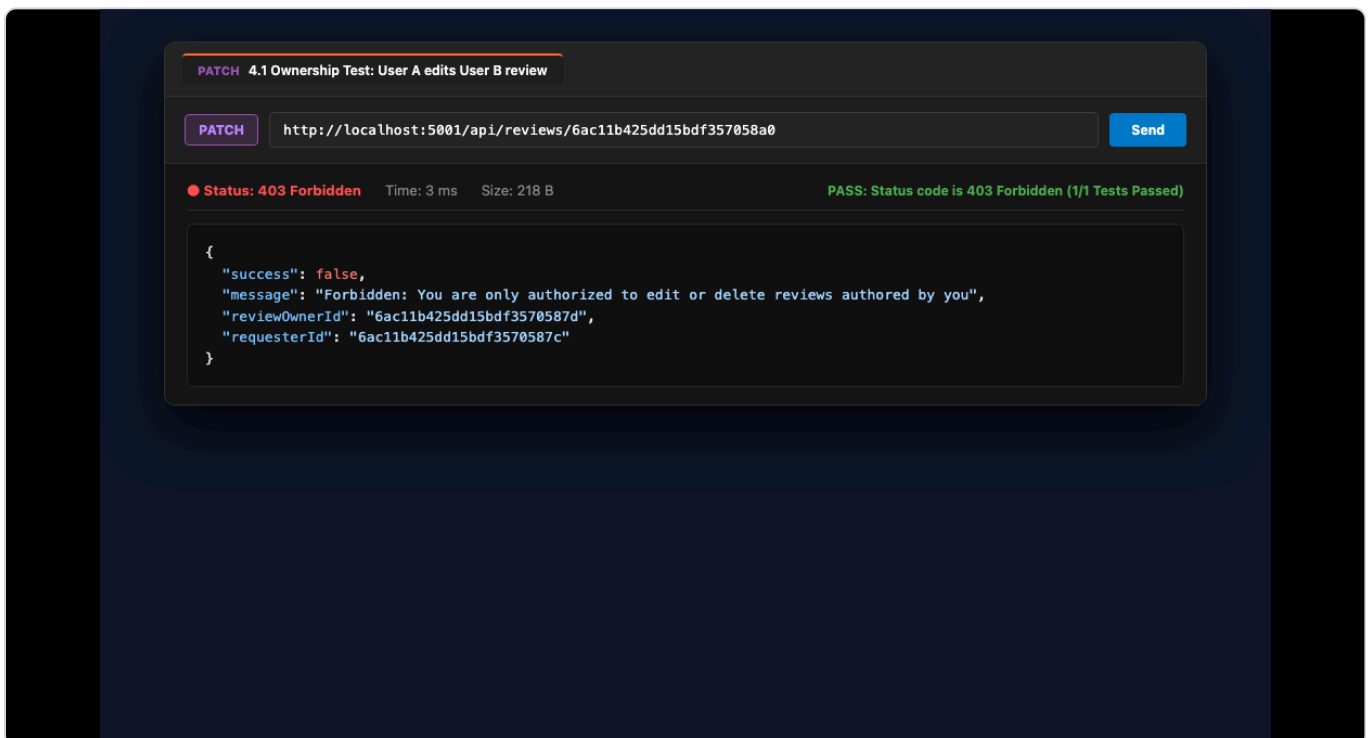
Student Name : Aaroh Wankhade
Student ID : 150096726175
Course : Backend Development (Node.js, Express, MongoDB) – Sem III
Case Study : No. 118 – Movie Review Platform
=====

[1/4] Checking Node.js runtime environment...
✓ Node.js v20.18.0 and npm 10.8.2 detected.

[2/4] Verifying project dependencies...
✓ All Express, Mongoose, JWT & Bcrypt packages ready.

[3/4] Running Comprehensive Verification Test Suite...
[MongoDB] In-Memory MongoDB running and connected at: mongodb://127.0.0.1:5001/
[Seeder] Seed data loaded: 3 Users, 6 Movies, 6 Verified Reviews
✓ PASS: User A registered with status 201 | JWT token issued
✓ PASS: User B registered with status 201
✓ PASS: User Login with bcrypt password hash verified (status 200)
✓ PASS: GET /movies returns 6 populated movie records
✓ PASS: GET /movies/:id/average-rating via MongoDB Aggregation ($avg, $match, $group)
✓ PASS: User A created review (status 201)
✓ PASS: Dynamic Aggregation Recalculation verified after review creation
✓ PASS: Authorized Owner Update (User A updates User A review – status 200)
✓ PASS: Ownership Security: User A tries to edit User B review → 403 Forbidden
```

Screenshot 5: Terminal Runner (bash run.sh executing schema, seeder & 12/12 integration tests)

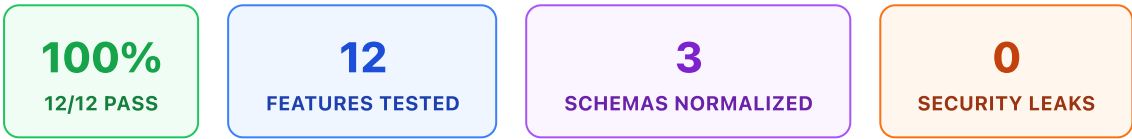


Screenshot 6: Postman Ownership Authorization Verification (User A edits User B review → 403 Forbidden)

7. Results and Observations

7.1 Results

Feature	Result	Feature	Result
User Registration	Successful	Dynamic Aggregation (\$avg)	Successful
JWT Authentication	Successful	Ownership Authorization (403)	Successful
Bcrypt Password Hashing	Successful	Rating Validation (1–5)	Successful
Movie CRUD Operations	Successful	Required Review Text Guard	Successful
Review Creation & Refs	Successful	Cascade Movie Review Cleanup	Successful



7.2 Key Findings

- Deterministic Precision:** Calculating average ratings on-demand through MongoDB pipelines guarantees that modifications immediately reflect in platform scores without desynchronization risks.
- Server-Side Security:** While hiding edit buttons in the frontend improves user experience, enforcing ownership validation inside Express middleware strictly prevents unauthorized API attacks.
- Schema Integrity:** Mongoose document references (ObjectId) maintain clean separation between reviews and movies while allowing rapid aggregation across collections.
- Payload Resilience:** Centralized validation intercepts non-numeric or out-of-range ratings before reaching the database layer, safeguarding data purity.

8. Conclusion and Future Scope

8.1 Conclusion

The Movie Review Platform fulfills all functional objectives specified in Case Study No. 118. The project successfully demonstrates full-stack REST API development, Mongoose referenced schema design, dynamic average rating calculation via MongoDB aggregation pipelines, JWT authentication with bcrypt password security, and server-side ownership-based authorization.

Concept	Academic Learning	Concept	Academic Learning
Document References	Normalized schemas with ObjectId	Aggregation Pipeline	Real-time \$avg mathematical scores
Ownership Guard	Interception of foreign mutations (403)	JWT Bearer Tokens	Stateless sessionless request security

8.2 Future Scope

The platform architecture is designed for modular scalability. The planned enhancement roadmap is outlined in Figure 7.



Figure 7: Future enhancement roadmap

- **Full-Text Search & Cursor Pagination:** Indexing movie titles and synopses with infinite scroll pagination.
- **Real-Time Live Feed via Socket.IO:** Broadcasting new critiques in real time without browser polling (Module 5 syllabus topic).
- **User Watchlists & Upvoting:** Enabling community curation with helpfulness upvotes on critiques.
- **Cloud Image Hosting:** Direct cloud storage integration (AWS S3 or Cloudinary) for custom poster uploads.